

UAV MOTION PLANNING HANDOUT

Dijkstra's Algorithm

Detailed derivation, implementation, correctness, and planning interpretation

Bijay Gaudel

Purpose. Given a weighted graph with nonnegative edge costs, Dijkstra's algorithm computes the least-cost path from one source to every reachable vertex. In motion planning, vertices represent collision-free configurations and edges represent feasible local motions.

1 Problem statement

Let $G = (V, E)$ be a finite directed graph. Each edge $(u, v) \in E$ has cost

$$c(u, v) \geq 0.$$

For a path $\pi = (v_0 = s, v_1, \dots, v_k = v)$, define

$$J(\pi) = \sum_{i=0}^{k-1} c(v_i, v_{i+1}).$$

The true shortest-path distance from source s to v is

$$\delta(s, v) = \min_{\pi: s \rightsquigarrow v} J(\pi),$$

with $\delta(s, v) = \infty$ if v is unreachable. The algorithm must return $g(v) = \delta(s, v)$ and a predecessor map from which a shortest path can be reconstructed.

For a grid planner, one common edge cost is Euclidean travel length,

$$c(u, v) = \|p_v - p_u\|_2,$$

possibly augmented by risk, energy, or clearance penalties. Dijkstra remains valid only if the resulting total edge costs are nonnegative.

2 From the Bellman equation to relaxation

Every shortest path to $v \neq s$ ends by entering v from some predecessor u . Therefore

$$\delta(s, s) = 0, \quad \delta(s, v) = \min_{(u,v) \in E} [\delta(s, u) + c(u, v)].$$

We do not know δ initially, so maintain tentative upper bounds $g(v)$:

$$g(s) = 0, \quad g(v) = \infty \quad (v \neq s).$$

Whenever an edge (u, v) is examined, perform *relaxation*:

$$\text{if } g(u) + c(u, v) < g(v), \quad g(v) \leftarrow g(u) + c(u, v), \quad \text{parent}(v) \leftarrow u.$$

The parent assignment records the final edge of the currently best known path. Because every finite $g(v)$ is the cost of an actual discovered path, it is always an upper bound:

$$g(v) \geq \delta(s, v).$$

3 Why the smallest tentative distance is safe

Dijkstra's key idea is to repeatedly finalize the unsettled vertex with minimum g .

Invariant 1 (Settled-vertex invariant). *Immediately after a vertex u is removed from the priority queue with the smallest valid key, $g(u) = \delta(s, u)$. Its distance never needs to change again.*

The intuition is a wavefront. With nonnegative edge costs, any route that first travels farther than the current wavefront cannot later become cheaper by adding more edges. This is precisely what fails when negative edges are allowed.

4 Detailed algorithm

The practical version below uses a binary min-heap and permits duplicate heap entries. This avoids needing a decrease-key operation. A popped entry is discarded when its key is stale.

Algorithm 1 Dijkstra with early goal termination and lazy heap deletion

Require: Graph $G = (V, E)$, nonnegative cost c , source s , optional goal t

Ensure: Distances g and predecessor map parent

```

1: for all  $v \in V$  do
2:    $g(v) \leftarrow \infty$ ;  $\text{parent}(v) \leftarrow \perp$ 
3:  $g(s) \leftarrow 0$ 
4:  $Q \leftarrow$  min-heap containing  $(0, s)$ 
5:  $S \leftarrow \emptyset$  ▷ settled vertices
6: while  $Q$  is not empty do
7:    $(d, u) \leftarrow \text{popMin}(Q)$ 
8:   if  $d \neq g(u)$  then ▷ stale heap entry
9:     continue
10:  if  $u \in S$  then
11:    continue
12:   $S \leftarrow S \cup \{u\}$ 
13:  if  $u = t$  then
14:    break ▷ the goal is now optimal
15:  for all  $(u, v) \in E$  do
16:     $d_{\text{new}} \leftarrow g(u) + c(u, v)$ 
17:    if  $d_{\text{new}} < g(v)$  then
18:       $g(v) \leftarrow d_{\text{new}}$ 
19:       $\text{parent}(v) \leftarrow u$ 
20:       $\text{push}(Q, (g(v), v))$ 
21: return  $g, \text{parent}$ 

```

Path reconstruction. If $g(t) < \infty$, start at t and repeatedly follow parent until s is reached; reverse the resulting list. If $g(t) = \infty$ after the queue empties, no graph path from s to t exists.

5 Correctness proof

Lemma 1 (Upper-bound property). *At every iteration, each finite $g(v)$ equals the cost of some discovered s - v path. Hence $g(v) \geq \delta(s, v)$.*

Proof. Initially only $g(s) = 0$ is finite and corresponds to the empty path. A relaxation sets $g(v) = g(u) + c(u, v)$, which appends edge (u, v) to the discovered path represented by $g(u)$. Thus every finite label remains the cost of a real path. No real path can cost less than the minimum over all paths, so $g(v) \geq \delta(s, v)$. $\square$

Theorem 1 (Dijkstra correctness). *When a vertex u is settled, $g(u) = \delta(s, u)$. Consequently, when the algorithm terminates, every reachable settled vertex has its exact shortest-path distance; if it stops when goal t is popped, the reconstructed s - t path is optimal.*

Proof. Assume for contradiction that u is the first settled vertex for which $g(u) > \delta(s, u)$. Consider a true shortest path P from s to u . Since s is settled first, P contains a first unsettled vertex y at the moment u is selected. Let x be the predecessor of y on P ; x is settled.

Because u is the first incorrectly settled vertex, $g(x) = \delta(s, x)$. When x was settled, edge (x, y) was relaxed, so

$$g(y) \leq g(x) + c(x, y) = \delta(s, x) + c(x, y) = \delta(s, y).$$

The upper-bound lemma gives $g(y) \geq \delta(s, y)$, hence $g(y) = \delta(s, y)$. Since y lies on a shortest path to u and all remaining edge costs are nonnegative,

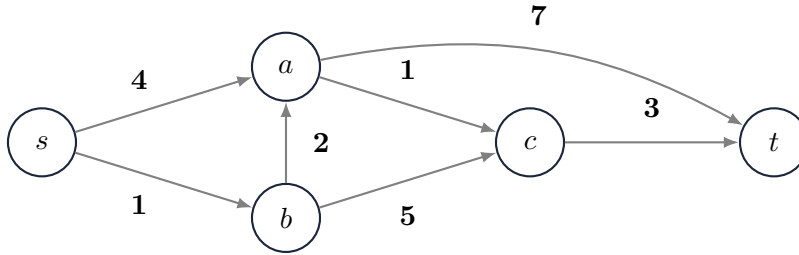
$$g(y) = \delta(s, y) \leq \delta(s, u) < g(u).$$

But y is unsettled and has a smaller queue key than u , contradicting the choice of u as the minimum. Therefore every settled label is exact. $\square$

Why nonnegative costs matter. Suppose $s \rightarrow a$ costs 2, $s \rightarrow b$ costs 5, and $b \rightarrow a$ costs -10 . Dijkstra settles a at cost 2, but the later route $s \rightarrow b \rightarrow a$ costs -5 . The settled-vertex invariant fails. Use Bellman–Ford or another method when negative edges are possible.

6 Worked example

Consider the graph below. Edge labels are costs.



Step	popped	$g(a)$	$g(b)$	$g(c)$	$g(t)$
0	—	∞	∞	∞	∞
1	s	4	1	∞	∞
2	b	3	1	6	∞
3	a	3	1	4	10
4	c	3	1	4	7
5	t	3	1	4	7

The predecessor chain is $t \leftarrow c \leftarrow a \leftarrow b \leftarrow s$, so the optimal path is

$$s \rightarrow b \rightarrow a \rightarrow c \rightarrow t, \quad J^* = 1 + 2 + 1 + 3 = 7.$$

Notice that a is first discovered with cost 4, then improved to 3 by relaxing (b, a) . A tentative label is not final until its vertex is popped with the minimum valid key.

7 Complexity and data structures

Let $|V|$ be the number of vertices and $|E|$ the number of directed edges.

Priority structure	Time	Typical use
Array / linear scan	$O(V ^2 + E)$	Dense or very small graphs
Binary heap	$O((V + E) \log V)$	Standard sparse planning graphs
Fibonacci heap	$O(E + V \log V)$ amortized	Mainly theoretical advantage

Memory is $O(|V| + |E|)$ when the graph is explicitly stored, plus $O(|V|)$ for distances, parents, and queue state. In an implicit grid, neighbors can be generated on demand, avoiding explicit storage of all edges.

For a d -dimensional grid with q cells per coordinate,

$$|V| = q^d.$$

This is the curse of dimensionality: even a polynomial-time graph algorithm can be expensive because the discretized graph itself grows exponentially with dimension.

8 Dijkstra as a motion planner

1. **Discretize the free configuration space.** A vertex may represent a 2-D position, a 3-D voxel, or a richer state such as position, velocity, and heading.
2. **Generate valid local motions.** Add an edge only if the local transition is collision-free and satisfies the chosen kinematic or dynamic constraints.
3. **Assign nonnegative costs.** Distance-only planning uses path length. Multi-objective planning may use

$$c(u, v) = w_\ell \ell(u, v) + w_E E(u, v) + w_R R(u, v) + w_T T(u, v), \quad w_i \geq 0.$$

Normalize terms carefully so one unit does not dominate merely because of scale.

4. **Run Dijkstra and reconstruct.** The result is optimal on this graph, not automatically in continuous space.
5. **Post-process if necessary.** A grid path is often jagged. Shortcutting, spline fitting, minimum-snap optimization, and time parameterization may be required before flight.

Resolution completeness. On a finite graph, Dijkstra finds a path if one exists. Relating that statement to the continuous planning problem requires assumptions: a valid continuous path should have positive obstacle clearance, and the grid resolution and motion primitives must be fine enough to represent it.

9 Dijkstra, uniform-cost search, and A*

Dijkstra is uniform-cost search on a graph. A* changes only the queue key:

$$\underbrace{g(n)}_{\text{cost from start}} \longrightarrow \underbrace{f(n) = g(n) + h(n)}_{\text{estimated solution cost}}.$$

Setting $h \equiv 0$ makes A* exactly Dijkstra. If h is admissible, $h(n) \leq h^*(n)$, A* remains optimal. A consistent heuristic additionally satisfies

$$h(n) \leq c(n, n') + h(n')$$

for every edge, which makes f nondecreasing along paths and avoids reopening closed vertices in the usual graph-search implementation. Dijkstra has no directional goal information; its wavefront expands in all low-cost directions. A good heuristic reduces expansions while preserving the optimality guarantee.

10 Implementation checklist and common errors

- Do not mark a vertex final when it is first discovered; finalize it only when popped with the smallest valid key.
- Handle stale heap entries or implement a correct decrease-key operation.
- Stop at the goal only when the goal is popped, not when it is first inserted.
- Use ∞ initialization and guard against numeric overflow in fixed-width integer implementations.
- Reject negative edge costs. Zero-cost edges are allowed.
- Store predecessors only when a strict improvement occurs; define a tie policy if deterministic paths matter.
- Collision-check every motion primitive, not only its endpoint.
- State the guarantee precisely: shortest path on the constructed graph under the selected edge-cost model.

11 Summary

The entire algorithm follows from one safe greedy decision:

With nonnegative costs, the unsettled vertex of minimum g can be finalized.

Relaxation propagates upper bounds, the priority queue selects the next provably optimal vertex, and parent pointers recover the path. The method is complete and optimal on a finite graph, but its practical cost depends strongly on graph size and resolution. A* retains the same logic while using a heuristic to focus the search toward the goal.